Data Preprocessing and Visualization

Load data • Clean & transform • Aggregate & group • Visualize insights

pandas

From raw data to meaningful insights!

1. Data Loading, Storage, and File Formats

- Reading and writing data in text format
- Binary data formats (e.g., pickle, parquet)
- Interacting with HTML and web APIs
- Interacting with databases

CSV JSON HTML DB Pickle Parquet

1.1 Read and Write Text Files

Text file (CSV)

```
import pandas as pd
df = pd.read_csv('data.csv')   # read
df.to_csv('output.csv', index=False)   # write
```

1.2 Read Binary Format

Binary file (Parquet)

```
import pandas as pd
df = pd.read_parquet('data.parquet')
```

1.3 Interact with Web API (JSON)

Web API (JSON)

```
import requests
url = 'https://api.example.com/data'
response = requests.get(url).json()
print(response)
```

JSON

1.4 Connect to Database

Database (SQL)

```
import sqlite3
conn = sqlite3.connect('data.db')
df = pd.read_sql('SELECT * FROM table', conn)
print.close()
```

2. Data Wrangling: Clean, Transform, Merge, Reshape

- Combining and merging data sets
- Reshaping and pivoting
- Data transformation
- String manipulation

Clean → Transform → Merge → Reshape

2.1 Combining and Merging Data Sets

Merge data sets

```
import pandas as pd
df1 = pd.read_csv('sales1.csv')
df2 = pd.read_csv('sales2.csv')
merged = pd.merge(df1, df2, on='id')
print(merged.head())
```

id	name	sales
1	Alice	100
2	Bob	150
3	Charlie	200

2.2 Reshaping and Pivoting

Pivot table (reshape data)

```
pivot_df = df.pivot_table(
    index='category',
    columns='month',
    values='sales', aggfunc='sum')
print(pivot_df)
```

Month	Jan	Feb	Mar
A	100	120	130
B	200	220	250

2.3 Data Transformation

Transform data (types, values)

```
df['price'] = df['price'].apply(lambda x: x*1.1)
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
```

2.4 String Manipulation

Work with text data

```
df['name'] = df['name'].str.strip()
df['name'] = df['name'].str.lower()
df['email'] = df['email'].str.split('@').str[0]
```

3. Data Aggregation and Group Operations

- Group by Mechanics
- Data Aggregation
- Groupby Operations and Transformations
- Pivot Tables and Cross Tabulation

Group by → Aggregate & Analyze

3.1 Group By Mechanics

Group data by category

```
grouped = df.groupby('department')
print(grouped.size())
```

department	count
HR	10
IT	15
Sales	12

3.2 Data Aggregation

Compute statistics

```
agg_df = df.groupby('department')[
  'salary'].agg(['mean','sum'])
print(agg_df)
```

department	mean	sum
HR	50000	500000
IT	70000	1050000
Sales	60000	720000

3.3 Groupby Transformations

Apply transformations

```
df['rank'] = df.groupby('department')['salary'] \
   .rank(ascending=False)
print(df[['department','salary','rank']])
```

3.4 Pivot Tables and Cross Tabulation

Pivot / Crosstab

```
pivot = pd.pivot_table(df, values='sales',\
index='region',columns='product', aggfunc='sum')
print(pivot)
```

Product	A	B	C
North	100	200	150
South	120	180	160

4. Data Visualization

matplotlib seaborn pandas

- A Brief matplotlib API Primer
- Statistical Visualization with Seaborn
- Plotting Functions in pandas
- Time Series
- Real-world case study applications

Turn data into insights!

4.1 Matplotlib Primer

Basic plotting with matplotlib

```
import matplotlib.pyplot as plt
plt.plot(df['date'], df['sales'])
plt.title('Sales Trend')
plt.show()
```



4.2 Seaborn Visualization

Statistical plots (with seaborn)

```
import seaborn as sns
sns.boxplot(x='category',y='sales',data=df)
plt.show()
```



4.3 Plotting Functions in pandas

Quick plots with pandas

```
df.plot(kind='bar', x='product', y='sales')
plt.show()
```



4.4 Time Series

Time-based analysis

```
df.set_index('date', inplace=True)
df.resample('M').mean().plot()
plt.show()
```



4.5 Case Study

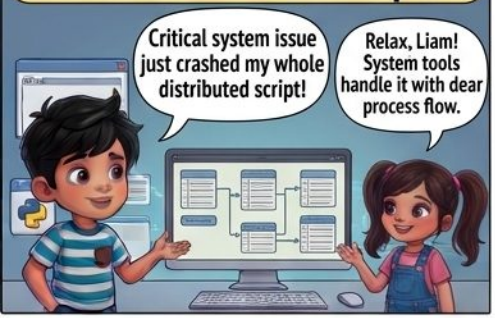
- Sales analysis
- Customer behavior
- Forecasting
- Business insights

Real-world application

Load → Clean → Transform → Analyze → Visualize → Get Insights

= Better data Better decisions!

1. Introduction to Data Prepr & Visualization Concepts



Data Preprocessing Flow

Pre-processing Flow → Data Storage → API/Command Calls → Aggregated Result

- Aggregates
- Transforms
- Selection
- Output

Processing Command	Usage
np.mean()	Data aggregation
requests	external API access
sqlalchemy	Database integration
psycopg2	External DB driver

Processing Tools	Usage
numpy.save	Array serialization
requests	Text file writing
sqlalchemy	Raw data loading
psycopg2	External DB driver

```
import numpy as np;
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```



2. Working with Data Operations & Commands



Preprocessing Tasks and Examples

Cleaning (DROPNA) | Transforming (MEAN) | Selection (GROUPBY) | Output (FILLNA)

Clean Data → SQL API → SQL → DB → Query Results

Task	Description
read_csv()	Loading standard CSV
requests	external API access
pickle	Persistent local saving
json	Token object representation

SQLAlchemy ORM

Python Classes → SQL → Query Results

```
f=open('f.txt','w');
np.savetxt(f, arr);
f.close()
```



3. Dataframe Operations: Data Cleaning vs. Data Analysis



Integration & Publishing (User event drive flow)

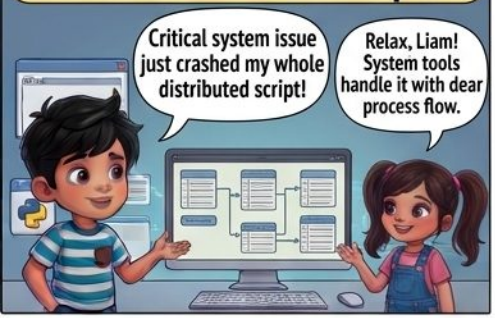
Advanced Ufunc Features & Performance	
Features	Description
Vectorization	(Binary data)
Multi-threading	Fast processed array persistence flow binary arrays data handled durability safe data flow /logical workflow

Feature	Tools
numpy.save	Array serialization
open('w')	Text file writing
np.fromfile	Raw data loading
pickle	Object persistence

Correct? We build robust routines by leveraging binary serialization, multi-treading, and performance profiling!

High performance + efficient workflows = powerful and reliable data handling!

1. Introduction to Data Prepr & Visualization Concepts



Data Preprocessing Flow

Pre-processing Flow → Data Storage → API/Command Calls → Aggregated Result

- Aggregates
- Transforms
- Selection
- Output

Processing Command	Usage
np.mean()	Data aggregation
requests	external API access
sqlalchemy	Database integration
psycopg2	External DB driver

Processing Tools	Usage
numpy.save	Array serialization
requests	Text file writing
sqlalchemy	Raw data loading
psycopg2	External DB driver

```
import numpy as np;
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```



2. Working with Data Operations & Commands



Preprocessing Tasks and Examples

Cleaning (DROPNA) | Transforming (MEAN) | Selection (GROUPBY) | Output (FILLNA)

Clean Data → SQL API → SQL → DB → Query Results

Task	Description
read_csv()	Loading standard CSV
requests	external API access
pickle	Persistent local saving
json	Token object representation

SQLAlchemy ORM

Python Classes → SQL → Query Results

```
f=open('f.txt','w');
np.savetxt(f, arr);
f.close()
```



3. Dataframe Operations: Data Cleaning vs. Data Analysis



Integration & Publishing (User event drive flow)

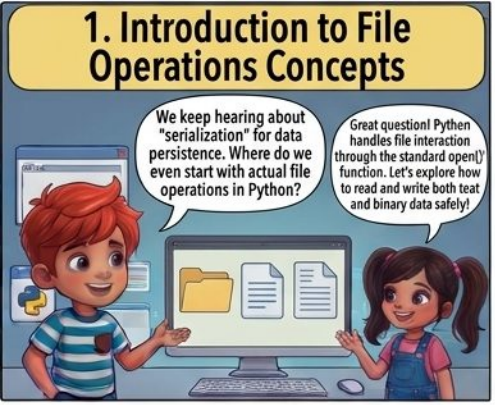
Advanced Ufunc Features & Performance	
Features	Description
Vectorization	(Binary data)
Multi-threading	Fast processed array persistence flow binary arrays data handled durability safe data flow /logical workflow

Feature	Tools
numpy.save	Array serialization
open('w')	Text file writing
np.fromfile	Raw data loading
pickle	Object persistence

Correct? We build robust routines by leveraging binary serialization, multi-treading, and performance profiling!

High performance + efficient workflows = powerful and reliable data handling!

1. Introduction to File Operations Concepts



Python File Basics: Reading and Writing

Text Format (Reading & Writing)

Text files are perfect for human-readable documents, configuration settings, or raw JSON data.

```
# Writing to a text file
data = _text.file

# Reading from a text file
data = request('dat', 'w")

result = np.meaan(huge_newaan("hello.txt')
```

(Output Diagram: Terminal prints content of "data.txt": Hello, World)

Binary Format (Durable & Persistent)

Binary files store raw machine-optimized bytes, suitable for images, data arrays, or serialized objects.

```
import numpy np;
data = numpy.np;

# Reading from a text file
for reachikzed, rawbytes, images,arrays:
    data = .result.load(f)
print(ont)
```

Image | bytes object

2. Working with Data Operations & Commands (File Modes)



File Workflow Diagram (Data input to Array logic)

Input Data | Parser/Decoder (e.g., CSV, JSON, bytes) | Array / Object (e.g., Numpy array, dict)

Input Data → Loaed to-Array Workflow → Integration Tools

Task	Description	Integration tool
		Common Built-in operators: np ops: np.add(), np.subtract() np file reading: np.fromfile() np advanced: np.save(), np.load()

```
f=open('f.txt','w');
np.savetxt(f, arr);
f.close()
```



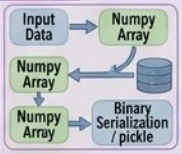
3. Dataframe Operations: Data Cleaning vs. Data Analysis (File Persistence)



File Workflows & Integrity (data persistent logical durability analysis)

Advanced File Operations & Workflow Durability

Cell Type	Description
open(...., 'w')	Text file writing
open(..., 'wb')	Binary file writing
pickle.dump()	Python object persistence
json.dump()	Token object persistence
%timeit	



Comparison Table (Text vs. Binary handling)

Operation Type	Description / Usage	Efficiency
Text Files	Human-readable, config, logs, JSON	High readability, low machine performance
Binary Files	Machine-optimized, raw bytes, images, arrays	Low readability, maximum machine performance

```
f=open('data.bin', 'wb');
pickle.dump(large_obj, f);
f.close()
```

```
import pickle
with open('model.pkl', 'rb') as f:
    loaded_model = pickle.load(f);
print(loaded_model)
```

Right file format + standard Python built-ins = powerful and reliable data handling!

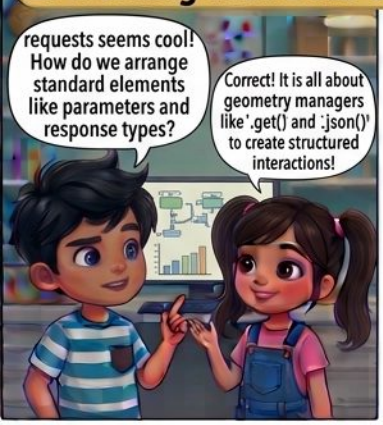
INTERACTING WITH HTML, WEB APIs & DATABASES

1. Exploring Web Interacting Concepts





2. Working with Data Operations & Commands (Workflow)



File Workflow Diagram (Data input to Array logic)

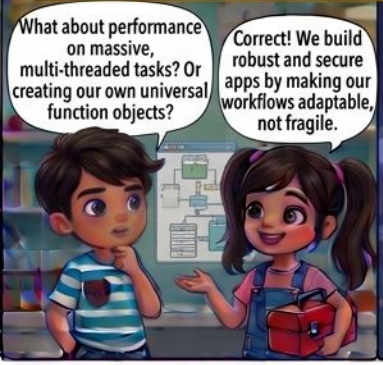
Input Data	Parser/Decoder (e.g., CSV, JSON, bytes)	Array / Object (e.g., Numpy array, dict)
Input Data →	Loaed to-Array Workflow →	Integration Tools

Task	Description	Integration tool
		Common Built-in operators: np ops: np.add(), np.subtract() np file reading: np.fromfile() np advanced: np.save(), np.load()

```
f=open('f.txt','w');
np.savetxt(f, arr);
f.close()
```



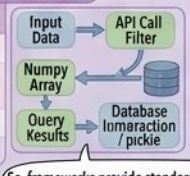
3. Unpacking Advanced Database Topics & Workflows



Workflow & Performance (query flow analysis)

Advanced Database Features & Performance

Cell Type	Description
Object-Relational	Mapping (SQLAlchemy)
multi-threading	Parallel query execution (multi-threading)
Indexing & Caching	(efficient queries)
rules	Database integrity rules



Integration

Operation Type	Description / Usage	Efficiency
requests	Raw web access	
beautifulsoup4	Web content extraction	
sqlalchemy	SQL logic/database integration	
psycopg2	PostgreSQL connection driver	

```
Tree aqlatchemy import create_engine
angtne = create_angine('sqlTo://:memory:');
connection = angine.connect()
vosult = connection.execute("SELECT * FROM users");
for row in roqult:
    prswt(rou)
```

So, frameworks provide standard primitives. Management tools handle event-objests and resources, and a good design integration tools ensures custom logic and performance optimization. Monitoring is key!

Flexible structures + efficient handling = powerful and reliable interacting with data sources!

DATA WRANGLING: CLEAN, TRANSFORM

1. Exploring Web Interacting Concepts





2. Data Wrangling Workflow & Commands (Filtering & Transforming)



Workflow & Commands (Transformation Logic)

Input Data → Parsing → Data Quality → Array / Object (o.g., Numpy array, dict)

Input Data → Loaed to-Array Workflow → Integration Tools

Task	Description
Operations	Description
dropna()	Removal of missing values
fillna()	Filling of missing values
replace()	Replacing specific values

Integration tool

Common Built-in operators:
np ops: np.add(), np.subtract()
np file reading: np.fromfile()
np advanced: np.save(), np.load()

```
f=open('f.tat','w');
np.savetxt(f, arr);
f.close()
```



3. Unpacking Advanced Database Topics & Workflows



Workflow & Optimization (performance analysis)

Data Cleaning & Manipulation

Feature	Description	Tools
dropna	Missing Data Removal	df.dropna
fillna	Missing Data Imputation	df.fillna
apply	Transformation & Lambda functions	df.apply
groupby	Aggregated transformations	df.groupby

```
def clean_text(text):
    return text.lower();

df['column'] = df['column'].apply(clean_text)
```

messy Input Data → Analysis → performance measurement + Optimization techniques → processed reliable relihniques

```
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```

Thought bubble and ꜱ ꜱnooerved context box only colored contest term processed logitis, reliability depenability columns, and safle, reliability dependable reliablt logiccally handled.

Final yellow summary box text: English, preserved logic and text and re-colored context. 😊

COMBINING AND MERGING DATA SETS, RESHAPING AND PIVOTING

1. Exploring Data Combining Concepts





2. Working with Data Operations & Commands (Workflow)



Workflow & Commands (query flow analysis)

Input Data → Parsing → Data Quality → Array / Object (o.g., Numpy array, dict)

Input Data → Loaed to-Array Workflow → Integration Tools

Task	Description
pd.merge()	Merging data sets
pd.concat()	Combining data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data

Integration tool

Common Built-in operators:
np ops: np.add(), np.subtract()
np file reading: np.fromfile()
np advanced: np.save(), np.load()

```
f=opsn('f.tat','w');
np.savetat(f, arr);
f.close()
```



3. Unpacking Advanced Merge & Reshape Topics & Workflows



Workflow & Performance (query flow analysis)

Advanced Data Combining Features & Performance

Cell Type	Object-Relational Mapping (SQLAlchemy)
df1	Parallel join execution (multi-threading)
df2	Indexing & Caching (efficient merges)
df1	Data merging integrity rules

```
import pandas as pd
df1 = pd.DataFrame{'id': [1,2], 'name': ["4', 'B'1};
df2 = pd.DataFrame{'id': [3,5], 'salory': 130, 20]};
result = pd.merge(3f1, df2, oh='id', how='outer')
print(result)
```

Input Data → Parsing / Decoupling → Merge Join (on keys) → Reshaping Logic → Query Results

requests	Web API access
beautifulsoup4	Content extraction
sqlalchemy	SQL logic/integration
psycopg2	DB connection driver

Thought bubble and s snooerved context box only colored contest term processed logiüs, reliability depenability columns, and salle, reliability dependable reliablc logicsaily handled.

Final summary yellow box text: Flexible structures + efficient handling = powerful and reliable data combining and reshaping!

DATA TRANSFORMATION: CLEANING AND RE-SHAPING

1. Exploring Data Transformation Concepts





2. Working with Data Operations & Commands (Workflow)



Workflow & Commands (query flow analysis)

messy Data → Parsing → Data Quality → Array / Object (o.g., Numpy array, dict)

Input Data → Data Transformation → Reshaping Logic

Task	Description
pd.merge()	Merging data sets
pd.concat()	Combining data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data

Integration tool

Common Built-in operators:
np ops: np.add(), np.subtract()
np file reading: np.fromfile()
np advanced: np.save(), np.load()

```
import pandas as pd
df = df.dropna().nelt();
print(cleaned)
```



3. Unpacking Advanced Data Transformation Topics & Workflows



Workflow & Optimization (performance analysis)

Advanced Data Transformation Features & Performance

Mapping (Serislization)	Object-Relstional Mapping (SQLAlchemy)
np.apply_along_axis	Numpy multi-dimensional transform
%timeit	Performance profiling
pickle	Persistent local saving
json	Token object representation

messy Data → Analysis → performance measurement → Optimization techniques → Query Results

```
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```

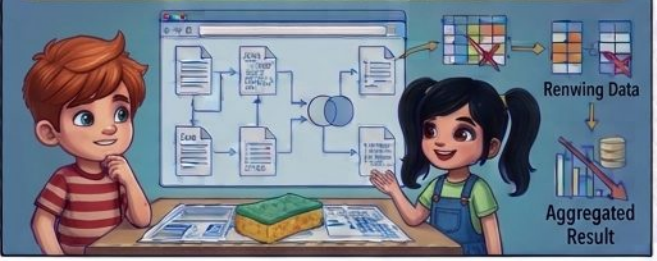
```
def transform(o):
    return x * 10;
arr = np.arange(30);
res = np.apply_along_axis(transform, 0, arr)
```

Thought bubble and s saooerved context box only solored contest terms precessed logiüs, reliability depenability columns, and sallo, reliability dopendable reliubk logicssily handled.

Flexible structures + efficient handling = powerful and reliable data transformation!

STRING MANIPULATION BASICS

1. Exploring String Manipulation Concepts





2. Working with Data Operations & Workflows (String Processing)



Workflow & Commands (query flow analysis)



Task	Description
str.replace()	Replacing substrings
str.split()	Splitting text
str.split()	Splitting text
str.join()	Joining strings

Integration tool

Common Built-in operators:
np.ape, np.add(), np.subtract()
np.file reading: np.loadfile()
np.advanced: np.save(), np.load()

```
import pandas as pd
df = df.dropna().melt();
print(cleaned)
```



3. Unpacking Advanced Data Transformation Topics & Workflows (String Logic)



Workflow & Optimization (performance analysis)

Common String Operations & Techniques	
str.extract()	Regex-based data removal
str.cat()	Concatenation transformations
str.cat()	Concatenation transformations
str.contains()	Sub-string logic/filtering
str.slice()	Tokenization and string slicing




```
huge_array = np.arange(10000000);
result = np.mean(huge_array)
```

```
def transform(e):
    return x * 10;
arr = np.arange(60);
res = np.apply_along_axis(transform, 0, arr)
```

Thought bubble and recovered context bonanly acloral contpon tome pleased logiis, reliability dependability columns, and path, reliability dependable reliobk logically handled.

Flexible structures + efficient handling = powerful and reliable data handling! 😊

DATA AGGREGATION AND GROUP OPERATIONS BASICS

1. Exploring Groupby Operations and Aggregation





2. Working with Data Operations & Commands (Workflow)



File Workflow Diagram (Data input to Array logic)



Task	Description
apply	Transformation & custom logic
%timeit	Performance measurement
pickle	Persistent local saving
json	Token object representation

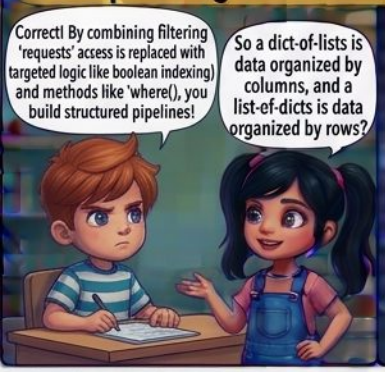
Integration tool

- Mapping (Serialization)
- Object-Relational Mapping (SQLAlchemy)

```
result = df.groupby('key')
.apply(oy_custom_func)
```



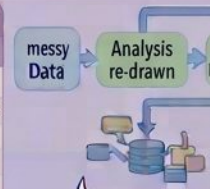
3. Unpacking Advanced Data Transformation Topics & Workflows (String Logic)



Advanced Database Features & Performance

Common String Operations & Techniques	
df.groupby()	Gregex specific
df.agg()	Transformation & custom logic
df.agg()	Performance measurement
df.transform()	Persistent local saving
df.filter()	Token object representation

```
result = df.groupby('key').apply(my_custom_func)
df.agg()
df.transform()
df.filter()
```



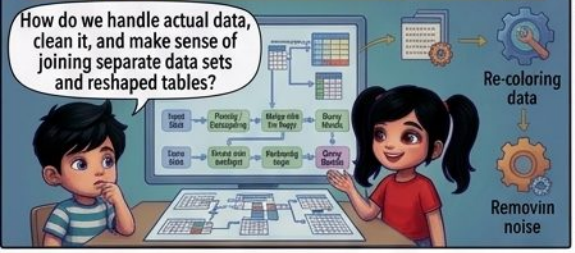
requests	Web API access
beautifulsoup4	Content extraction
sqlalchemy	SQL logic/integration
psycopg2	DB connection driver

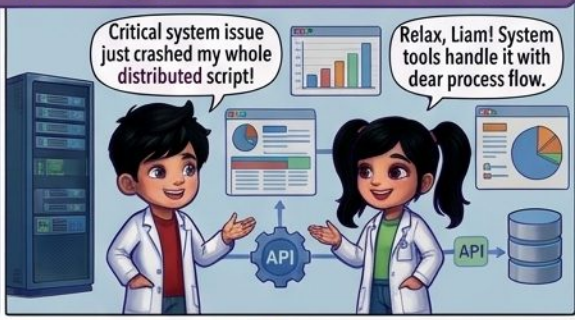
So framework provide standard primitives. Management tools handle event-objects and resources, and a good design integration tools ensures custom logic and performance optimization. Monitoring is key!

Flexible structures + efficient handling = powerful and reliable data handling!

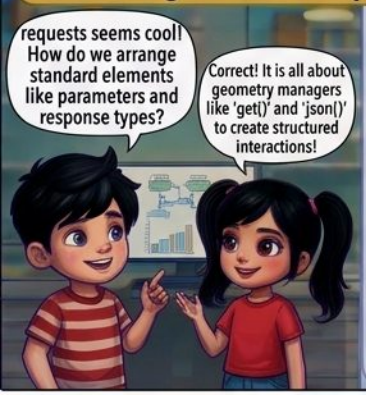
PIVOT TABLES AND CROSS TABULATION

1. Exploring Data Combining Concepts

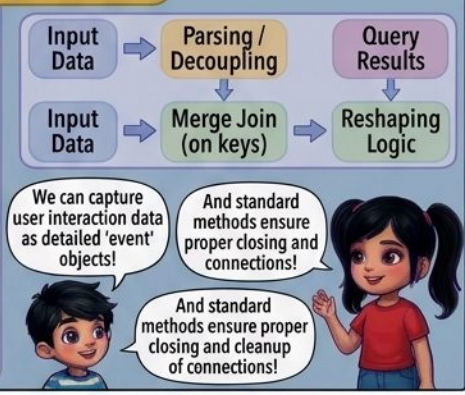




2. Working with Data Operations & Commands (Workflow)



Commands	Usage
pd.merge()	Merging data sets
pd.concat()	Combining data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data
pd.concat()	Combining data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data
pd.melt()	Geometry layouts
pd.melt()	Structured layouts



3. Unpacking Advanced Merge & Reshape Topics & Workflows



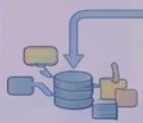
Advanced Data Combining Features & Performance

Cell Type	Description
Object-Relational Mapping	Object-Relational Mapping (SQLAlchemy)
df.agggli)	Parallel join execution (multi-threading)
Indexing & Caching	Indexing & Caching (efficient merges)
df.filter	Data merging integrity rules

```
import pandas as pd;
df1 = pd.DataFrame({'id': [1, 2], 'name': ['A', 'B']});
df2 = pd.DataFrame({'id': [2, 3], 'salary': [10, 20]});
result = pd.merge(df1, df2, on='id', how='outer');
print(result)
```

Workflow & Performance (query flow analysis)





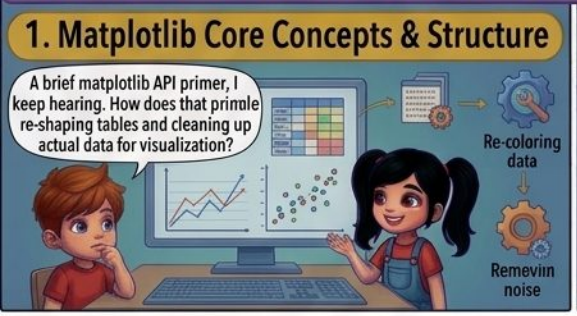
Integration	
requests	Web API access
beautifulsoup4	Content extraction
sqlalchemy	SQL logic/integration
psycopg2	DB connection driver

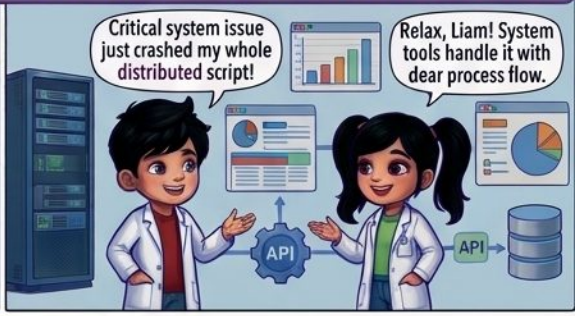
Final thought provide: low preserved with bull threaded take tables and mstions with thought! 💡

Pivot tables + cross tabulation = powerful categorical summaries! 😀

DATA VISUALIZATION: A BRIEF MATPLOTLIB API PRIMER

1. Matplotlib Core Concepts & Structure



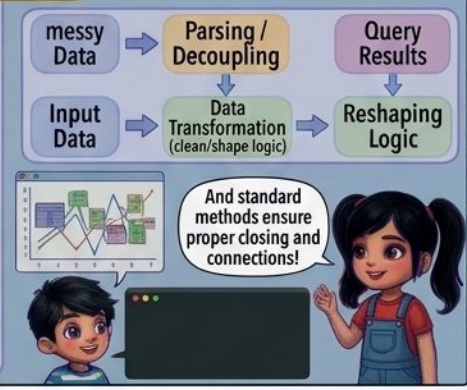


2. Matplotlib Workflows & Commands (Workflow)



Commands	Usage
pd.merge()	Merging data sets
pd.concat()	Combining data
pd.melt()	Unpivoting data
pd.melt()	Unpivoting data
pd.melt()	Structured layouts

```
import matplotlib.pyplot as plt;
x=[1,2,3]
y=[4,5,6]
plt.plot(x,y)
plt.show()
```



3. Unpacking Advanced Merge & Reshape Topics & Workflows

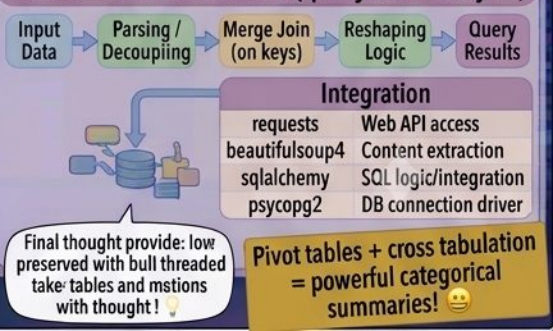


Advanced Data Combining Features & Performance

Cell Type	Description
apply where	apply where
contex	pd operators: pd.add(), pd.subtract()
pd data reading	pd.fromfile()
pd advanced	pd.save(), pd.load()

```
import pandas as pd;
df1 = pd.DataFrame({'id': [1, 2], 'name': ['A', 'B']});
df2 = pd.DataFrame({'id': [2, 3], 'salary': [10, 20]});
result = pd.merge(df1, df2, on='id', how='outer');
print(result)
```

Workflow & Performance (query flow analysis)



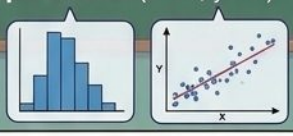
PYTHON: STATISTICAL VISUALIZATION & DATASET PLOTTING

1. PANDAS DATASET PLOTTING & BASIC CHARTS

```
import pandas as pd
df = pd.read_csv('data.csv')
df.plot.hist()

df.plot.scatter(x='X', y='Y')
```

pandas `df.plot` is an entry point to matplotlib. Quick & easy plotting of pandas DataFrames.



2. INTRODUCING SEABORN: A HIGH-LEVEL INTERFACE

Seaborn makes attractive statistical graphics with concise code. Uses pandas automatically!

```
import seaborn as sns
sns.set_theme()
sns.lineplot(data=df, x="X", y="Y")
```

2. PANDAS DATASET PLOTTING & BASIC CHARTS

Try **hist, bar, line, scatter**! It's convenient for a quick look.

Wow, it is so pretty and and easy to style! Much more high-level.

Understand the trend between two numerical variables.

sns.jointplot shows the bivariate relationship & individual distributions.

See the overall density and the separate stats!

3. STATISTICAL FUNCTIONS: REGRESSION & BIVARIATE DISTRIBUTIONS

3a. FILE I/O AND & SHOWS

```
sns.regplot(data=df, x="A", y="B")
```

sns.regplot combines a scatter plot with linear regression modeling.

3b. FILE JOINTPLOT

```
jointplot
data=df,
x="Y",
kind="hex")
```

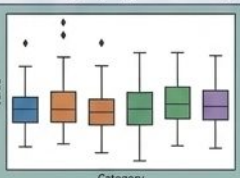
sns.jointplot shows the bivariate relationship & individual distributions.

append

Esseplions
Pile UG
Numanic Methods

4. ADVANCED DISTRIBUTIONS & CATEGORICAL PLOTS

```
sns.boxplot(data=df,
x="Category", y="Value")
```



sns.violinplot(data=df, x="Category", y="Value")



Plot Type	Use	Function
Box Plot	Compare Distribution	sns.boxplot
Violin Plot	Distribution & Density	sns.violinplot

Key seaborn Plot Types

KDE Plot	Univariate Density	sns.kdeplot

sns.boxplot shows the distribution of quantitative data across categories.

Compare ranges, medians ,alians, and outliers for groups!

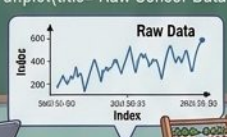
Context-specific plots for deeper insights.

PYTHON: TIME SERIES ANALYSIS & REAL-WORLD CASE STUDY APPLICATIONS

1. INTRO TO TIME SERIES: DATA STRUCTURES & BASIC OPERATIONS

```
import pandas as pd
df = pd.read_csv('sensor_data.csv',
  parse_dates=['date'], index_col='date')
df.plot(title='Raw Sensor Data')
```

PANDAS



Time Series is data indexed by time.

Check for missing dates

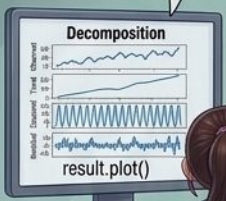
Resample data (e.g., daily to weekly).

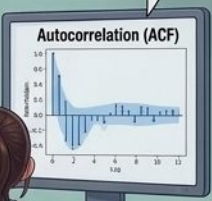
Shift data (lags). Rolling statistics.

2. EXPLORATORY ANALYSIS: DECOMPOSITION & AUTOCORRELATION

Decompose a series to isolate key components.

Use ACF to identify optimal lags for modeling.





Identify long-term movement, patterns, and noise.

Understand relationships with past values.

3. MODELING FUNCTIONS: ARIMIA & STATISTICAL APPROACHES

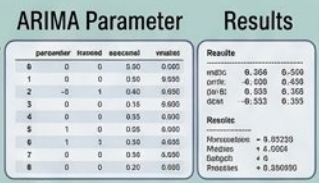
- Resample data (e.g., daily to we)
- Shift data (lags).
- Rolling statistics.

Data Prep → Feature Eng → Model selection

```
model = statsmodels.tsa.arima.model.
ARIMA(train, order=(p,d,q))
results = model.fit()
```

3a. UNIVARIATE MODELS

ARIMA Parameter

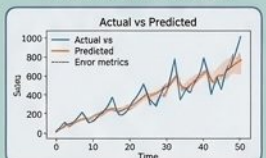


Results

Classic ARIMA (Auto-Regressive Integrated Moving Average) model.

3b. CASE STUDY: SALES FORECASTING

Actual vs Predicted

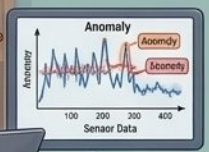


Forecast future sales to optimize inventory.

4. REAL-WORLD APPLICATIONS & CASE STUDIES

Predictive Maintenance

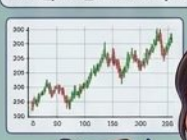
`anomaly_detection_model.predict(new_data)`



Predict equipment failure to reduce downtime.

Financial Forecasting

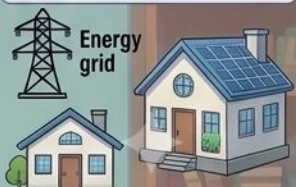
`stock_price_forecaster.predict(future_dates)`



Analyze stock market trends to inform investment decisions.

Energy Load Prediction

`grid_load_forecaster.predict(time_of_day)`



Predict peak demand for efficient energy distribution.